Interactive Real-Time Task Web App

JavaScript, jQuery, DOM, Events, Storage and AJAX

THEORY AND CASE STUDY | PRACTICAL 12

Aim

To build an interactive real-time web application using JavaScript and jQuery, demonstrating DOM manipulation, event handling, local storage, and AJAX integration.

Software/Tools Required

- HTML5 for task forms, filters, metrics, and application structure.
- CSS3 for the responsive task board and interactive states.
- JavaScript ES6+ for application data and client-side behavior.
- jQuery 3 for DOM manipulation, delegated events, and $.ajax() requests.
- Browser localStorage for persistence; VS Code and Live Server for development.

Theory

1. jQuery DOM manipulation and event handling

jQuery selectors locate page elements, while methods such as empty(), append(), text(), and html() update the interface. Event handlers respond to form submissions, clicks, and checkbox changes. Delegated handlers on a stable parent also work for list items created later.

2. localStorage persistence

localStorage stores string values in the browser across page reloads. JSON.stringify() serializes task objects before saving, and JSON.parse() restores them. A storage key keeps the application data organized.

3. AJAX and REST APIs

$.ajax() sends asynchronous HTTP requests without reloading the page. Success handlers process JSON responses and update the interface; error handlers communicate request failures. A secondary local JSON resource can provide data when the remote service is unavailable.

4. State-driven rendering

Application state is the current task collection and selected filter/search query. After a user action or imported response updates that state, a renderer rebuilds the visible task list and refreshes total, completed, and pending counts.

Practical 12 | Case Study and Implementation

Case Study Overview

The SmartTask case study is a task-management web application. Users can create tasks, mark them complete, remove them, filter by completion status, and search titles or categories. Tasks persist in localStorage. AJAX actions import tasks from JSONPlaceholder or a bundled JSON resource, and a weather panel requests current conditions from Open-Meteo.

Key Features Implemented

- Add, complete, and delete tasks with jQuery event handlers.
- Filter tasks by all, pending, and completed status; search by text.
- Persist task state in localStorage and update live task metrics.
- Import JSON tasks asynchronously with a remote API and local JSON fallback.
- Request and display live weather through a jQuery AJAX GET request.
- Render loading, success, empty, and error feedback without a page reload.

Core Implementation Details

HTML Structure (index.html)

- Provide the task form, filter and search controls, dynamic task list, metrics, AJAX actions, and weather status area.
- Load jQuery before the application script.

CSS Styling (style.css)

- Use a responsive grid and accessible visual states for tasks and controls.
- Style pending/completed tasks, priorities, filters, and AJAX feedback.

JavaScript Functionality (app.js)

- Load and save the task collection as JSON in localStorage.
- Use jQuery to render task elements, update statistics, and bind form, filter, search, checkbox, and delete events.
- Use $.ajax() to request weather and import remote JSON tasks; fall back to tasks.json when the remote task request fails.
- Update application state, persist changes, and rerender the UI after user actions and AJAX responses.

Conclusion

The SmartTask app combines jQuery-based DOM updates and event handling with persistent browser storage and asynchronous API requests. Its task operations, search and filters, live statistics, and AJAX imports demonstrate how client-side features can work together without full-page reloads.